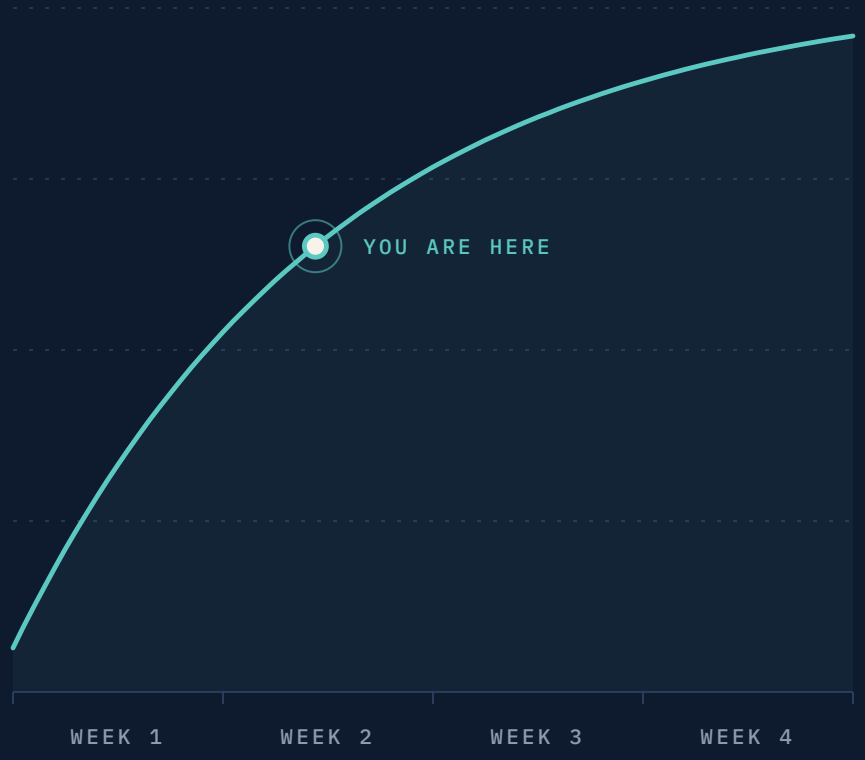
WEEK 2 · DAY 2, PART 1 · OVERVIEW

pandas

Series and DataFrames, Part 1



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 5 SECTIONS / 5 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Tenth Note

Yesterday's file came back without its tickers. Today the book is two hundred bonds in holdings.csv. First put names on our four bonds, and check the coupon income still ties when the other system sends the faces in its own order, and when it leaves one out. Then load the file: how big, which columns, anything missing. Show me the spreads, and pull up CUSIP 99039NAC0 and its price.

ABOUT 50 MIN WITH EXERCISES / 5 SECTIONS / 5 EXERCISES

Your Route Through Day 2, Part 1

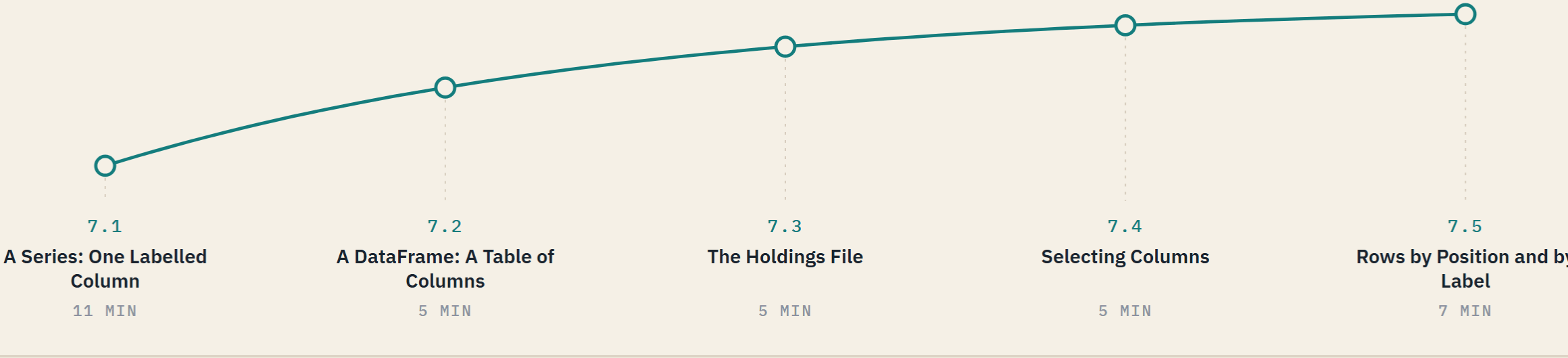


FIGURE 1 Five sections, in order, with about how long each takes.

Time. About 33 minutes for the lesson, then about 15 for the five exercises.

7.1 A Series: One Labelled Column

prices: an index of labels beside an array of values

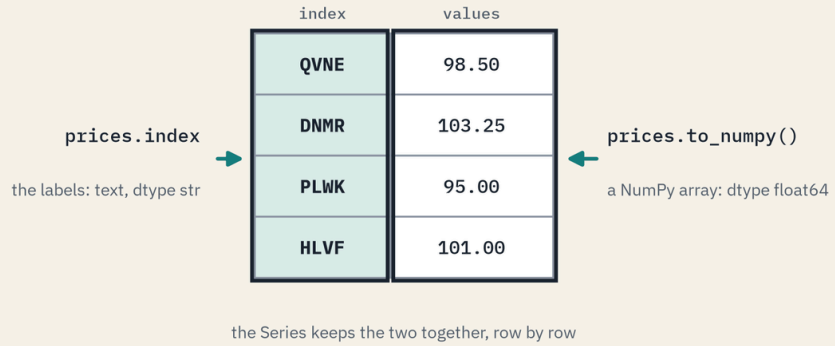


FIGURE 2 The prices Series in two parts: an index of four tickers, and its values, a NumPy array of float64.

LABEL FOUR PRICES

```
print(prices)
```

```
QVNE     98.50
DNMR    103.25
PLWK     95.00
HLVF    101.00
```

WHAT IS INSIDE

```
print(prices.to_numpy())
print(type(prices.to_numpy()))
```

```
[ 98.5  103.25  95.   101.  ]
<class 'numpy.ndarray'>
```

The values are a NumPy array, so NumPy's arithmetic still works.

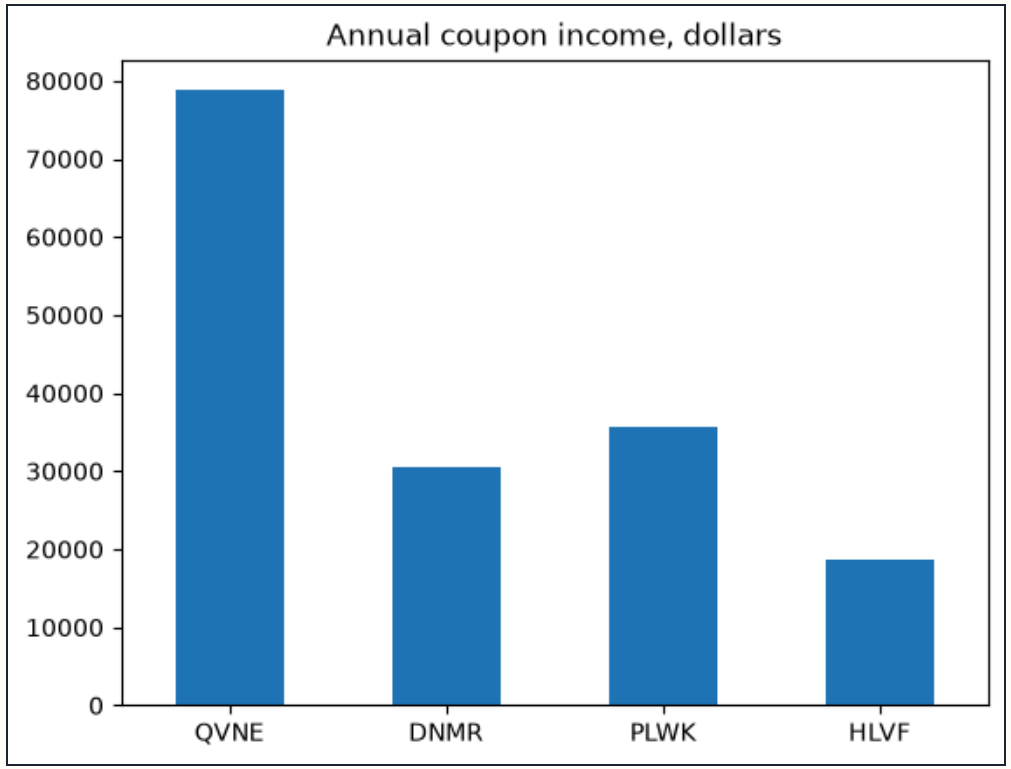
Income by Ticker

THE INCOME OF EACH POSITION

```
incomes = faces * coupons / 100
print(incomes)
print('Total annual coupon income:', incomes.sum())
```

```
QVNE    78750.0
DNMR    30625.0
PLWK    35625.0
HLVF    18750.0
dtype: float64
Total annual coupon income: 163750.0
```

A SERIES DRAWS ITSELF



163,750: week 1's number, a ticker on every line.

7.2 A DataFrame: A Table of Columns

desk: six Series sharing one index

index	Ticker	Coupon	Maturity	Price	Face	Rating
0	QVNE	5.250	2033	98.50	1500000	BBB
1	DNMR	6.125	2034	103.25	500000	BB+
2	PLWK	4.750	2031	95.00	750000	A-
3	HLVF	7.500	2032	101.00	250000	B+

desk['Price'] →

0	98.50
1	103.25
2	95.00
3	101.00

Name: Price, dtype: float64

one type per column; one index for the whole table

FIGURE 4 The desk table as six columns on one shared index; the Price column lifted out is a Series.

PULL OUT A COLUMN

```
desk_prices = desk['Price']
print(desk_prices)
print(type(desk_prices))
```

```
0     98.50
1    103.25
2     95.00
3    101.00
Name: Price, dtype: float64
<class 'pandas.Series'>
```

Everything in 7.1 works on a column.

7.3 The Holdings File

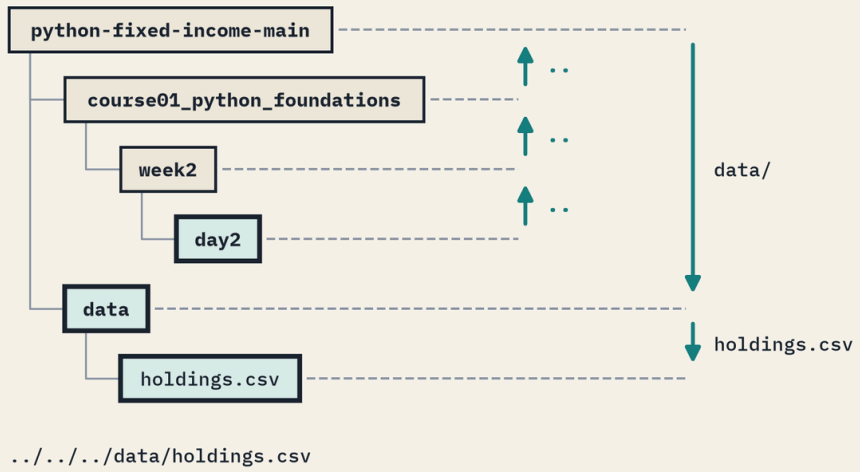


FIGURE 5 From day2, three steps marked dot dot climb to the repo; the path then goes down into data to holdings.csv.

LOAD THE FILE

```
holdings = pd.read_csv(data_path)
```

THE FIRST ROWS

	AS_OF_DATE	CUSIP	TICKER
0	2026-09-30	99080CAE8	BEZE
1	2026-09-30	99049XAE2	BEZH
2	2026-09-30	99039NAC0	BEZL
3	2026-09-30	99039NAG1	BEZL

BEZL holds two bonds: a ticker is not a bond.

7.4 Selecting Columns

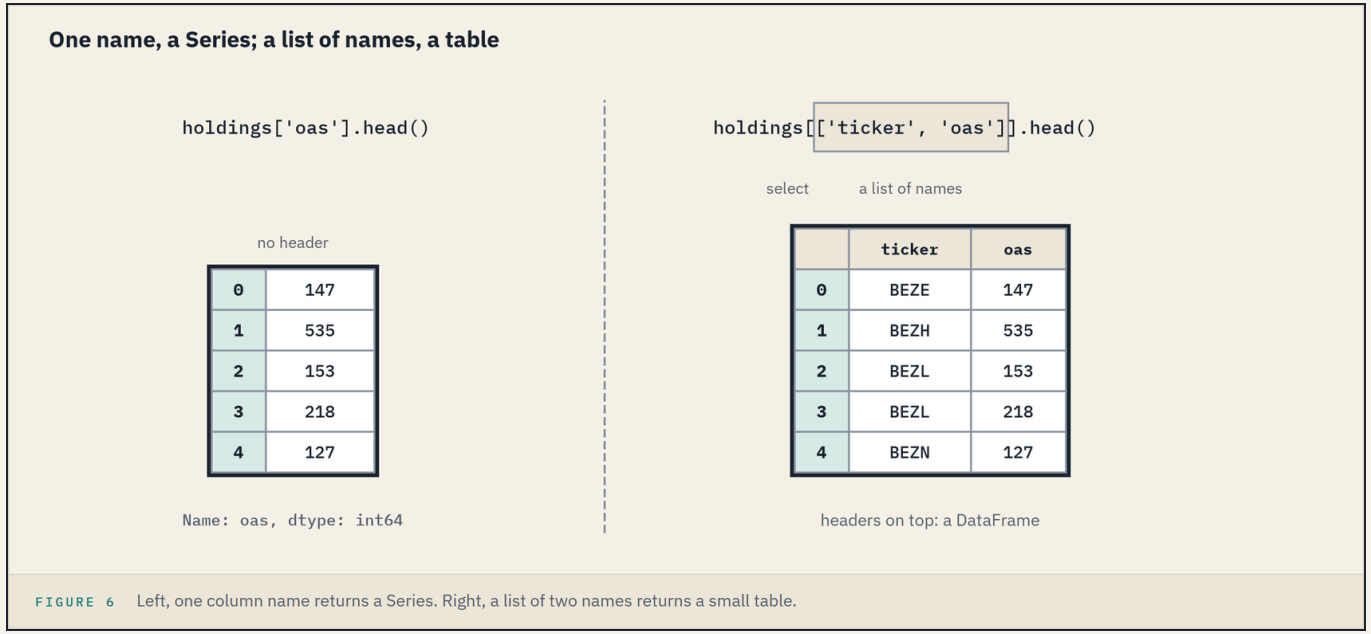


FIGURE 6 Left, one column name returns a Series. Right, a list of two names returns a small table.

ONE NAME

```
holdings['oas'].head()
0    147
1    535
2    153
```

A LIST OF NAMES

	TICKER	COUPON	MATURITY	PRICE
0	BEZE	7.000	2032-06-18	106.727
1	BEZH	8.125	2029-04-17	97.556

One name, a Series; a list of names, a table.

7.5 Rows by Position and by Label

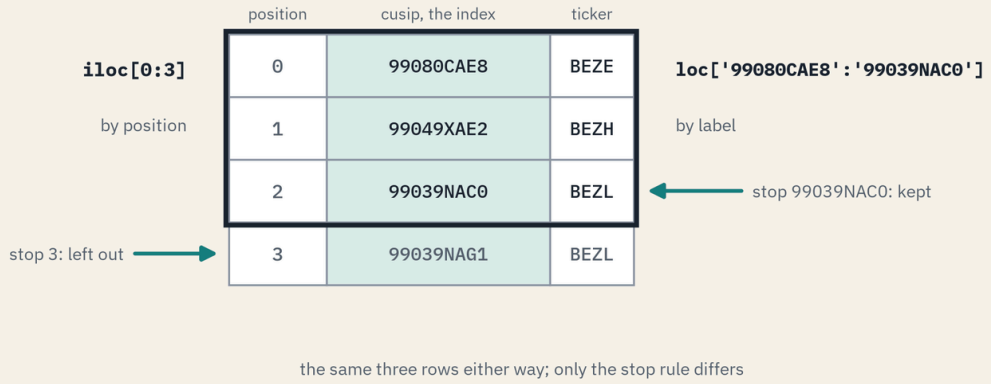


FIGURE 7 iloc from 0 to 3 stops before position 3; loc stops on its last CUSIP. Both frame the same three bonds.

A BOND BY ITS CUSIP

```
print(by_cusip.loc['99039NAC0'])
print(by_cusip.loc['99039NAC0', 'price'])
```

```
as_of_date          2026-09-30
ticker                    BEZL
issuer          Bezoquil Freight
sector           Transportation
rating                       BB
coupon                    5.125
maturity             2030-06-03
price                     98.73
```

`iloc` stops before its end; `loc` keeps it, as in 7.1.

Lookups in Excel and pandas

EXCEL SIDE BY SIDE

QUESTION	PANDAS	EXCEL
The third row	`holdings.iloc[2]`	row 4 of the sheet, under the header
The first three bonds, first four columns	`holdings.iloc[0:3, 0:4]`	`=TAKE(A2:S201, 3, 4)`, Microsoft 365
Price of a CUSIP	`by_cusip.loc['99039NAC0', 'price']`	`=VLOOKUP("99039NAC0", B:I, 8, FALSE)`
The same, without counting columns	`by_cusip.loc['99039NAC0', 'price']`	`=INDEX(I:I, MATCH("99039NAC0", B:B, 0))`
A CUSIP that is not there	`KeyError`	`#N/A`

TWO BONDS, BY LABEL

```
print(by_cusip.loc[['99080CAE8', '99039NAC0'], ['ticker', 'price']])
```

A lookup that fails loudly is the one a desk wants.

You Can Now

- Build a Series and a DataFrame from lists, a dictionary, Series, or rows.
- Say what an index is, and why pandas lines two Series up by label.
- Look for **`NaN`** after arithmetic on two sources, before trusting a total.
- Load a holdings file and take a first look: its size, columns, types, and anything missing.
- Pull out the columns a question needs: one name for a Series, a list for a table.
- Look a bond up by its CUSIP with **`loc`**, and pick rows and columns by position with **`iloc`**.

NEXT
WEEK 2 · DAY 2, PART 2

pandas: Series and DataFrames, Part 2

Summarizing, filtering, sorting, and changing a portfolio of two hundred bonds, and showing a table well.

